A Theory-Guided LLM Pedagogical Agent for STEM+C Scaffolding Without Dependence

Supplementary Materials

Overview

This supplement enumerates the prompt features, inputs, and outputs for Copa's multi-agent architecture, along with additional detail on how each agent functions and operationalizes theory. Raw prompt text is maintained separately in the repository[1]; this document provides an interface-level specification suitable for replication.

Shared Task Context Features

All agents are prompted with a consistent task description. Table 1 enumerates the task-level constants and constraints that appear in the prompts, using the example 1-D XYZ truck task.

Table 1: Shared task context features (Truck Task).

Feature	**Definition / Value**
Task goal	Construct a computational model of a truck that accelerates from rest, cruises at a speed limit, and decelerates to stop at a stop sign.
Acceleration bounds	± 4 m/s^2.
Speed limit	15 m/s.
Initial position	-60 (environment units).
Stop sign position	38.16.
Time step constant	`DeltaT = 0.1`.
State variables	`x_position`, `x_velocity`, `x_acceleration`.
Constants	`DeltaT`, `SpeedLimit`, `StopSignPosition`.
Execution model	Initialization logic under `When Green Flag Clicked` block; iterative updates under `Simulation Step` block.
Lookahead distance (ground truth)	28.125 (students must compute using kinematic equations; agents are instructed not to provide this value directly to students).
Testing tools	Students can run by clicking the Green Flag; Graph/Table tools may be used to visualize variable changes over time.
Runnable model structure	Two principal block groups: (1) `[When Green Flag Clicked]` for initialization; (2) `[Simulation Step]` for the loop.
Expert reference	One correct reference model is provided for logical equivalence checking (presented on the next page).

[1] `https://anonymous.4open.science/r/BJET26_Supplementary_Materials-5F5B/README.md`

Expert reference model, translated from the block-based code on screen to textual form for the LLM by parsing the raw logs.

```
[When Green Flag Clicked]
Set DeltaT to 0.1
Set SpeedLimit to 15
Set x_position to -60
Set x_velocity to 0
Set x_acceleration to 4

[Simulation Step]
Change x_velocity by (x_acceleration * DeltaT)
Change x_position by (x_velocity * DeltaT)
If x_velocity > SpeedLimit then
    Set x_acceleration to 0
If StopSignPosition - x_position < 28.125 then
    Set x_acceleration to -4
If x_position > StopSignPosition and x_velocity < 0 then
    Set x_velocity to 0
    Stop Simulation
```

The **Learner Model** is also shared across all four agents and is presented in Table 2.

Table 2: Shared learner model components used across Copa's agents.

Component	**Definition / Value**
Student query	The students' current chat message to Copa.
Computational model	Current state of the students' computational model.
Task context	The phase or subtask the students are currently working on.
Recent actions	Most recent XYZ environment actions, used to infer strategy.
Current strategy	Students' current cognitive strategy inferred from recent actions.
Strategy history	Previously inferred strategies stored in the learner model.
Physics mastery	Percent of physics rubric points earned.
Computing mastery	Percent of computing rubric points earned.
Overall mastery	Percent of total task points earned.
Current learner state	The students' current inferred learner state.
Past learner states	Previous learner states used as historical context for state updates.
Past dialogue states	The last three dialogue states observed in the interaction history.
Domain knowledge	Retrieved domain knowledge relevant to students' current need.

Each time students perform an action in the XYZ environment, the learner model updates the recent actions, task context, computational model state, and mastery scores accordingly. Similarly, when students send a query to the agent, dialogue states and policies are stored alongside the student query, agent feedback, and conversation history. The KnowledgeAgent and StrategyAgent alternate asynchronous probes every 30 seconds to keep the learner model updated with relevant domain knowledge and strategy history. Each time the StrategyAgent finishes running, the AssessmentAgent is invoked to infer a new learner state from the most recent strategy and add it to the learner model.

LLM Selection and Prompt Engineering

We evaluated LLMs from the Gemini, Claude, and GPT families and selected GPT for optimal performance. Asynchronous Evidence agents use the high-reasoning capabilities of *GPT-5*, while the synchronous DialogueAgent employs *GPT-5-Chat* to support low-latency interaction. Prompts were developed using a human-in-the-loop procedure from prior work [anonymized for peer review], which uses chain-of-thought prompting to ground LLM reasoning in rubric-aligned evidence and few-shot examples.

With CoTAL, initial few-shot examples are selected to include both clear ground-truth cases and "sticking point" cases that reveal common scorer disagreements, with chain-of-thought rationales linking quoted student evidence to rubric criteria. During iterative refinement, active learning targets persistent validation errors by adding new high-value examples that address the model's recurring mistakes and sharpen the prompt over time. Prompts were refined across four hour-long sessions with 10-15 researchers, who interacted with Copa using XYZ and provided feedback that informed subsequent revisions.

Agent 1: StrategyAgent (Evidence Module)

Purpose

Infer students' cognitive strategy from sequences of modeling actions in XYZ.

Functionality

In prior work, we used a cognitive task model to map low-level XYZ environment actions to model-building constructs such as constructing, debugging, and assessing (a superset of the action vocabulary discussed shortly). Using sequence-mining methods, we derived productive and unproductive strategies (also discussed shortly), which capture linear combinations of successive actions.

The StrategyAgent receives the past ten XYZ environment actions, summarizes the sequence in terms of the strategies observed, and outputs one final strategy label to update the student strategy history in the learner model.

Theoretical Motivation

Cognitive strategies are considered across a wide variety of learning theories (e.g., Social Cognitive Theory, Self-Regulated Learning, Cognitive Load Theory, etc.), motivating the need for tracking *how* students approach problem solving in the learner model.

In our case, through the lens of Social Cognitive Theory, learners use cognitive strategies not only to build domain knowledge but also to structure problem solving, with growing skill in these strategies strengthening self-efficacy.

Action Vocabulary (Defined in the Prompt)

Table 3: Defined student action types used for strategy inference.

Action	Definition
`BUILD`	Add or connect runnable blocks affecting model execution.
`ADJUST`	Modify, remove, or reposition existing runnable blocks.
`EXECUTE`	Run the model using the Green Flag.
`DRAFT`	Add or modify disconnected (non-runnable) blocks.
`VISUALIZE`	Use graph or table tools to inspect model behavior.

Strategy Labels (Defined in the Prompt)

Table 4: StrategyAgent cognitive strategy labels. Thumbs up/down refer to good/bad strategies.

Strategy	Operational Definition
`DEPTH_FIRST_ENACTING` 👎	Extended `BUILD`/`ADJUST` sequences with no model execution.
`TINKERING` 👍	Frequent alternation between model edits and executions.
`TOOL_USE` 👍	High frequency of `VISUALIZE` actions during modeling.
`RUN_REPEAT` 👎	Three or more consecutive `EXECUTE` actions without edits.
`DRAFTING` 👍	Using disconnected blocks as placeholders, i.e., commenting code.

Input

A sequence of the students' last 10 XYZ environment actions.

Output Schema

{"summary" : string, "strategy" : one of Table 4}

Agent 2: AssessmentAgent (Evidence Module)

Purpose

Infer the dyad's learner state from model structure, strategy, and recent history.

Functionality

The AssessmentAgent receives XYZ task information, the current state of the computational model, the current strategy, and past learner states (discussed shortly), then generates a summary of this information to infer a new learner state for the learner model update.

Theoretical Motivation

Personalized support requires the agent to reason about learner state rather than respond only to immediate task context. From an Evidence-Centered Design perspective, learner state serves as the evidentiary bridge between observable student data and pedagogical action, allowing the system to transform logs, dialogue, and model state into interpretable inferences about students' current knowledge, strategies, and needs. In EDF, this motivates learner-state modeling as the basis for agent adaptivity: the downstream DialogueAgent uses learner state to personalize what support to provide and how that support should respond to the student's evolving problem-solving process.

Learner State Labels (Defined in the Prompt)

Table 5: Learner state definitions used by the AssessmentAgent.

Learner State	Definition
STARTING	No runnable blocks present in the student model.
PROGRESSING	Incomplete but logically correct model using effective strategies.
EXPLORING	Incomplete but correct model using suboptimal strategies.
DEBUGGING	Model contains at least one logical error.
STRUGGLING	Persistent debugging or lack of progress.
FINISHED	Model is complete and without error.

Input

Current computational model, current strategy, past learner states.

Output Schema

{"summary" : string, "learner_state" : one of Table 5}

Agent 3: KnowledgeAgent (Evidence Module)

Purpose

Identify a proxy for the students' Zone of Proximal Development (ZPD), determine the next step the students need to take in the environment, and select the domain knowledge to retrieve for scaffolding.

Functionality

The KnowledgeAgent takes the student's current computational model as input, compares it to an expert model to estimate the ZPD, and identifies the logical next step needed for progress (e.g., updating velocity at each simulation step). It then determines the domain knowledge most relevant to that step (e.g., updating variables, the velocity kinematic equation, loops) and generates a retrieval recommendation. This recommendation drives RAG-based retrieval from the knowledge base, and the retrieved domain knowledge is stored in the learner model as context to guide subsequent student-agent interactions.

With regard to *how* the KnowledgeAgent determines the ZPD, we make two assumptions: (1) students know and can do only what is reflected in the current state of their model; and (2) with support, students can complete the full model, since the XYZ curriculum and computational modeling tasks are designed for the target age group and aligned with NGSS standards. Under these assumptions, if the agent can accurately identify the student's current model state and the expert target state, then the gap between the two serves as a proxy for the ZPD.

These assumptions may break down in some cases, such as when students make model progress through successful but uninformed trial-and-error that outpaces their conceptual understanding. However, the open-ended nature of XYZ makes such cases difficult to sustain, and our analyses over the past several years suggest that they occur infrequently.

Theoretical Motivation

The KnowledgeAgent is explicitly instructed to (1) "[identify] which parts of [the students'] model are missing or incorrect," (2) "[determine] the immediate next step needed in their Zone of Proximal Development (ZPD)," and (3) "[recommend] domain knowledge retrieval (one sentence) that maps clearly to the knowledge base via semantic search."

ZPD defines the gap between what a learner can do independently and what they can do with guidance from a More Knowledgeable Other (MKO; Copa, in this case). In EDF, ZPD alignment matters because it grounds adaptive scaffolding in the learner's current model state. If support targets knowledge students already possess, the agent risks redundancy and over-reliance; if it targets knowledge too far beyond the student's current understanding, it risks confusion and reduced self-efficacy. By estimating the next reachable step and retrieving the domain knowledge needed for that step, the KnowledgeAgent supports guidance that remains proximal to the learner's current progress and better positions the student to internalize the skill and later perform it independently.

Input

Student's computational model (i.e., their code) and the expert model presented earlier.

Output Schema

{`"summary"` : string, `"recommended_domain_knowledge"` : string}

Agent 4: DialogueAgent (Decision and Feedback Modules)

Purpose

The DialogueAgent is responsible for generating adaptive, student-facing feedback. It operates in two explicit stages: (1) dialogue state inference based on the current interaction context, and (2) feedback generation conditioned on the inferred state, learner evidence, and pedagogical policy constraints.

Functionality

The DialogueAgent acts as Copa's central orchestrator, synthesizing evidence from the other three agents to provide students with informed, personalized, and adaptive feedback. As soon as students ask Copa a question, it immediately retrieves information from the learner model, including current task context, physics/computing/overall mastery, current learner state, recent dialogue-state history, and retrieved domain knowledge. It then performs two stages of reasoning: first, it infers the student's dialogue state from the current query; second, it combines that state with the learner model to select one or two dialogue policies that define pedagogical intent. Dialogue states and policies are both discussed shortly.

These dialogue policies specify *how* Copa should respond. For example, the DialogueAgent may probe understanding, set a short-term goal, suggest a XYZ action, promote self-efficacy, provide domain knowledge, recommend a strategy, support engagement, seek help, or extend the student's current knowledge boundary. The prompt tightly constrains this process so that responses remain short, peer-like, and adaptive: probing is prioritized, support fades as mastery increases, students must explain *why* before Copa suggests implementation, and Copa must anchor feedback in the current computational model rather than move ahead of student progress.

After selecting a dialogue policy, the DialogueAgent generates the final student-facing talk move. This output is the only system component visible to students, allowing Copa to maintain conversational flow while the other agents reason asynchronously in the background. In this way, the DialogueAgent operationalizes both the Decision and Feedback modules of EDF: it translates learner evidence into pedagogical intent, then translates that intent into adaptive scaffolding during interaction.

Theoretical Motivation

The DialogueAgent prompt explicitly encodes both SCT and social constructivist principles. For SCT, the agent is instructed to promote self-efficacy by "[offering] praise for students' successes or encouragement when students struggle" and to ensure responses "use an upbeat and positive tone." The DialogueAgent is also instructed to "support goal-setting" and "prompt the students to decide on an intermediate goal to achieve." For social constructivism, the prompt requires the `PROBE_UNDERSTANDING` dialogue policy to ask "why" questions that prompt students to demonstrate understanding, states that students "must explain *why* before Copa suggests implementation," and emphasizes that responses should be phrased as "questions or encouragement, not explanations."

These instructions motivate learner-aware adaptation in the DialogueAgent. From an SCT perspective, the agent should respond not only to correctness, but also to students' confidence, persistence, and progress, using encouragement and fading support to sustain agency as mastery increases. From a social constructivist perspective, the agent should treat dialogue as the mechanism through which understanding is elicited and refined, prioritizing probing questions, student explanation, and student-led next steps over answer delivery. In EDF, these commitments translate

into adaptive talk moves that use the learner model and dialogue state to decide when to probe, when to encourage, when to suggest action, and when to withhold explanation so that students construct understanding through interaction.

Dialogue State Labels (Defined in the Prompt)

Table 6: Dialogue state definitions.

Dialogue State	Definition
OFF_TASK	Utterance unrelated to the task.
DISENGAGED	Minimal or apathetic response.
ENACTING	Student describing ongoing modeling actions.
FRUSTRATION	Expression of confusion or difficulty.
INFORMATION_SEEKING	Direct question about the XYZ task/domain concepts.
ACKNOWLEDGING	Student acknowledges prior feedback.
SUGGESTING	Student proposes a next action.
UNDERSTANDING_RESPONSE	Evidence of conceptual understanding.
NOT_UNDERSTANDING_RESPONSE	Evidence of misunderstanding.

Dialogue Policy Labels (Defined in the Prompt)

Table 7: Dialogue policy definitions.

Policy	Definition
PROBE_UNDERSTANDING	Ask a question to elicit reasoning.
SET_GOAL	Help students articulate a short-term objective.
SUGGEST_ACTION	Propose a concrete next modeling step.
PROVIDE_DOMAIN_KNOWLEDGE	Share relevant conceptual knowledge.
PROMOTE_SELF_EFFICACY	Encourage confidence and persistence.
STRATEGY_RECOMMENDATION	Suggest a more productive learning strategy.
SUPPORT_ENGAGEMENT	Re-engage off-task or disengaged students.
AWAIT_ACTION	Pause feedback while students act.
SEEK_HELP	Recommend external help after repeated failure.
END_GAME	Acknowledge task completion and conclude.
GET_CLARIFICATION	Request clarification (policy only; not a state).

Response Constraints

- Responses must be no more than 100 characters.
- Responses must maintain a collaborative peer persona and adhere to social-cognitive and social-constructivist principles (defined in the prompt).
- Hidden inputs (states, mastery, retrieved knowledge) must not be disclosed.

Input

The students' current chat message and learner model, including current computational model, current task context, current physics/computing/overall mastery, current learner state, past three dialogue states, and retrieved domain knowledge.

Output Schema

The DialogueAgent must return a JSON object with the following required fields:

```
{
  "dialogue_state_summary": string,
  "dialogue_state": dialogue state label,
  "dialogue_policy_summary": string,
  "dialogue_policy": one or two dialogue policy labels,
  "response": string
}
```